

RESUMEN TEORICO PARA PARCIALES

Este resumen integra el contenido principal de los PDFs de la carpeta para Desarrollo de Apps Mobile y Desarrollo Frontend. La idea es que sirva para estudiar teoria rapido, detectar temas fuertes y despues decidir que profundizar.

1. APIS, REST, HTTP, JSON Y POSTMAN

QUE ES UNA API

API significa Application Programming Interface o Interfaz de Programacion de Aplicaciones.

Una API es un mecanismo que permite la comunicacion entre sistemas o aplicaciones de forma organizada y estandarizada. Funciona como un intermediario entre un cliente y un servidor.

Se dice que una API funciona como un contrato porque define:

- que operaciones se pueden realizar
- como deben enviarse las solicitudes
- que datos deben enviarse
- como sera la respuesta

MODELO CLIENTE-SERVIDOR

El modelo cliente-servidor es la base del funcionamiento de internet y de la mayoria de las apps modernas.

- El cliente inicia la comunicacion. Suele ser una app web, una app mobile o un frontend.
- El servidor recibe la solicitud, procesa la logica y devuelve una respuesta.

Flujo basico:

1. El cliente realiza una solicitud.
2. El servidor recibe la peticion.
3. El servidor procesa la informacion.
4. El servidor devuelve una respuesta.
5. El cliente recibe y muestra el resultado.

Una buena analogia es la de un restaurante:

- el cliente es la aplicacion
- la API es el mozo
- el servidor es la cocina

La aplicacion no accede directamente a la base de datos ni a la logica interna del servidor:

pide algo a través de la API.

TIPOS DE APIS

Según el material, aparecen estos tipos:

- APIs de sistema: interactúan con sistema operativo o hardware.
- APIs de bibliotecas o frameworks: simplifican el uso de tecnologías complejas.
- APIs web: permiten intercambiar información por internet, normalmente usando HTTP.
- APIs públicas y privadas
- APIs para hardware

En mobile, las más importantes suelen ser las APIs web, especialmente REST + JSON.

REST Y SOAP

REST

REST (Representational State Transfer) es el modelo más usado para desarrollar APIs web modernas.

Características:

- utiliza HTTP
- trabaja normalmente con JSON
- modela la información como recursos
- cada recurso se identifica con una URL
- suele ser stateless
- es liviano y flexible

Se usa mucho en apps mobile y frontend porque simplifica la comunicación entre cliente y servidor.

SOAP

SOAP (Simple Object Access Protocol) es un protocolo más estricto.

Características:

- define reglas estrictas de intercambio
- utiliza normalmente XML
- suele ser más pesado y formal
- sigue usándose en sistemas corporativos críticos

DIFERENCIA GENERAL

- REST domina en apps web, frontend y mobile.
- SOAP aparece mas en entornos empresariales donde se priorizan reglas muy estrictas.

HTTP

HTTP es el protocolo de comunicacion usado constantemente en la web y en las APIs.

Los metodos HTTP principales son:

- GET: obtener informacion
- POST: crear o enviar informacion
- PUT: actualizar informacion
- DELETE: eliminar informacion

Relacion con CRUD:

- Create -> POST
- Read -> GET
- Update -> PUT
- Delete -> DELETE

REQUEST Y RESPONSE

Cuando un cliente usa una API:

- envia una request o solicitud
- recibe una response o respuesta

En esa comunicacion suelen viajar:

- metodo HTTP
- URL o endpoint
- headers
- body
- codigo de estado HTTP

CODIGOS HTTP

Los codigos de estado indican el resultado de la peticion.

- 2XX: exito
- 4XX: error del cliente
- 5XX: error del servidor

Ejemplos importantes:

- 400 Bad Request: la solicitud esta mal formada
- 401 Unauthorized: no hay autenticacion valida
- 403 Forbidden: el cliente esta autenticado pero no tiene permisos suficientes
- 500 Internal Server Error: error interno del servidor

JSON

JSON (JavaScript Object Notation) es el formato mas usado para intercambiar informacion entre aplicaciones y APIs.

Ideas clave:

- aunque su nombre viene de JavaScript, no depende de JavaScript
- es simple de leer y escribir
- es liviano
- es facil de procesar en muchos lenguajes
- por eso domina en APIs REST

ENDPOINT

Un endpoint es un punto especifico de acceso dentro de una API.

La URL identifica el recurso y el metodo HTTP indica la accion.

Ejemplo conceptual:

- GET /usuarios -> obtener usuarios
- POST /usuarios -> crear un usuario
- GET /usuarios/5 -> obtener un usuario especifico

POSTMAN

Postman es una herramienta para probar APIs sin tener que programar primero una interfaz.

Permite:

- enviar solicitudes HTTP
- probar endpoints
- enviar headers y body
- ver respuestas JSON
- verificar codigos HTTP
- detectar errores antes de integrar la API en la app

Uso basico:

1. Elegir metodo HTTP.
2. Escribir endpoint.
3. Agregar headers o body si hace falta.
4. Enviar la solicitud.
5. Analizar la respuesta.

2. JWT, AUTENTICACION Y AUTORIZACION

QUE ES JWT

JWT significa JSON Web Token.

Es un estandar abierto para transmitir informacion de forma segura y verificable entre distintas partes. Se usa mucho en:

- autenticacion
- autorizacion
- APIs REST
- aplicaciones web
- aplicaciones mobile
- microservicios

IDEA CENTRAL

Con JWT, el servidor genera un token firmado digitalmente y el cliente lo envia en peticiones posteriores para demostrar identidad y permisos.

Esto permite evitar, en muchos casos, el almacenamiento constante de sesiones en el servidor.

AUTENTICACION VS AUTORIZACION

Es una distincion obligatoria para parcial.

- Autenticacion: responde quien sos. Verifica identidad.
- Autorizacion: responde que podes hacer. Verifica permisos.

Frase corta que conviene memorizar:

- la autenticacion identifica
- la autorizacion otorga permisos

STATELESS

JWT se considera un mecanismo stateless.

Stateless significa sin estado: cada request contiene la informacion necesaria para que el servidor la procese sin depender de una sesion almacenada en memoria.

Ventajas:

- mayor escalabilidad
- menor dependencia de sesiones centralizadas
- mas facilidad para arquitecturas distribuidas
- desacople entre frontend y backend

MODELO TRADICIONAL: SESIONES Y COOKIES

Antes de JWT, lo habitual era autenticar asi:

1. El usuario inicia sesion.
2. El servidor valida credenciales.
3. El servidor crea una sesion.
4. La sesion se identifica con un SESSION_ID.
5. Ese identificador se envia al navegador mediante una cookie.
6. El navegador reenvia la cookie en solicitudes futuras.
7. El servidor busca la sesion asociada.

Ideas clave:

- HTTP es stateless
- las cookies ayudan a mantener contexto entre requests
- la cookie no suele guardar todos los datos del usuario, sino un identificador

Tipos de cookies:

- cookies de sesion: temporales
- cookies persistentes: duran mas tiempo

POR QUE APARECE JWT

El enfoque con sesiones funciona, pero presenta problemas cuando crecen la cantidad de usuarios, servidores o servicios.

JWT aparece como alternativa moderna porque:

- evita consultar sesiones continuamente
- simplifica sistemas distribuidos
- permite validar tokens en distintos servidores
- mejora escalabilidad

ESTRUCTURA DE UN JWT

Un JWT tiene tres partes:

- Header
- Payload
- Signature

Se suele ver así:

HEADER.PAYLOAD.SIGNATURE

HEADER

Contiene metadatos del token, por ejemplo:

- tipo de token
- algoritmo de firma

Campos habituales:

- typ: tipo de token, por ejemplo JWT
- alg: algoritmo utilizado para firmar

PAYLOAD

Contiene información del usuario o del proceso de autenticación.

Puede incluir:

- identificador del usuario
- fecha de emisión
- fecha de expiración
- roles
- permisos

Claims muy importantes:

- sub: sujeto o usuario identificado
- exp: fecha de expiración
- iss: emisor
- nbf: desde cuando el token es válido

Punto clave de examen:

- el payload no está cifrado

- se puede decodificar facilmente
- por eso no deben guardarse datos sensibles dentro del token

JWT garantiza:

- integridad
- autenticidad

JWT no garantiza por si solo:

- confidencialidad

SIGNATURE

La signature es la firma digital.

Sirve para verificar que el contenido del token no fue alterado.

El servidor:

1. firma el token al crearlo
2. recalcula la firma cuando lo recibe
3. compara el resultado

Si el token fue alterado, la firma deja de coincidir y el servidor lo rechaza.

INTEGRIDAD VS CONFIDENCIALIDAD

Esto suele confundirse mucho.

- Integridad: asegura que el contenido no fue modificado.
- Confidencialidad: asegura que el contenido no pueda leerse.

JWT se apoya en firma digital, no en ocultamiento.

Conclusion:

- JWT firma la informacion
- JWT no necesariamente la cifra

FLUJO BASICO CON JWT

1. El usuario envia credenciales.
2. El servidor autentica al usuario.
3. El servidor genera el JWT.
4. El cliente almacena el token.

5. El cliente envia el token en cada request protegida.
6. El servidor verifica firma, expiracion y datos.
7. Si es valido, permite acceso.

Header habitual:

Authorization: Bearer <token>

ACCESS TOKEN Y REFRESH TOKEN

ACCESS TOKEN

- se usa para acceder a recursos protegidos
- viaja normalmente en el header HTTP
- suele tener vida corta

REFRESH TOKEN

- sirve para obtener un nuevo access token sin volver a loguearse
- suele tener vida mas larga
- no deberia enviarse en cada request comun

Flujo:

1. El usuario inicia sesion.
2. El servidor entrega access token y refresh token.
3. La app usa el access token normalmente.
4. Cuando expira, usa el refresh token para pedir uno nuevo.

ALMACENAMIENTO SEGURO

En mobile y frontend, no solo importa usar JWT sino guardarlo correctamente.

Riesgos mencionados:

- robo de tokens
- almacenamiento inseguro
- uso de localStorage con exposicion a XSS

Buenas practicas:

- usar HTTPS
- usar expiracion corta para access tokens
- usar refresh tokens
- validar correctamente exp, iss, nbf

- no guardar datos sensibles en el payload
- proteger bien las claves de firma

ALGORITMOS DE FIRMA

El material menciona dos enfoques generales:

- simetrico: misma clave firma y valida
- asimetrico: clave privada firma y clave publica valida

Idea teorica:

- el enfoque asimetrico permite que varios servicios validen sin exponer la clave privada de firma

JWT VS OAUTH 2.0

Otra diferencia teorica importante:

- JWT es un formato de token
- OAuth 2.0 es un protocolo de autorizacion

Suelen usarse juntos, pero no son lo mismo.

IDEA FINAL PARA MEMORIZAR

JWT no es magia ni seguridad automatica. Es una herramienta potente para autenticacion y autorizacion modernas, pero depende de una implementacion correcta.

3. JAVASCRIPT

ORIGEN

JavaScript fue creado en 1995 en el contexto del crecimiento de la web. Brendan Eich lo implemento en muy poco tiempo. Primero se llamo Mocha, luego LiveScript y finalmente JavaScript.

En 1997 se definio el estandar ECMAScript.

En 2009, con Node.js, JavaScript paso a ejecutarse tambien fuera del navegador.

Desde ES6 (2015) incorporo mejoras importantes como let, const y funciones flecha.

QUE ES JAVASCRIPT

JavaScript es un lenguaje de programación que permite crear contenido dinámico e interactivo en páginas web.

Se usa para:

- validaciones
- animaciones
- manipulación del DOM
- eventos
- actualizaciones en tiempo real
- frontend
- backend
- mobile

CARACTERÍSTICAS PRINCIPALES

- alto nivel
- interpretado
- tipado dinámico
- tipado débil
- multiparadigma
- multiplataforma
- dinámico

ALTO NIVEL

Su sintaxis está orientada a facilitar lectura y escritura por parte de desarrolladores.

INTERPRETADO

El código se ejecuta en tiempo de ejecución mediante un intérprete o motor, sin requerir una compilación tradicional previa a lenguaje máquina.

TIPADO DINÁMICO Y DÉBIL

- una variable puede cambiar de tipo en tiempo de ejecución
- el lenguaje hace conversiones implícitas en algunos casos

MULTIPARADIGMA

Permite programar con distintos enfoques:

- imperativo
- funcional
- orientado a objetos

MOTORES DE JAVASCRIPT

Los motores son programas que analizan e interpretan código JavaScript.

Ejemplos:

- V8: Chrome y Node.js
- SpiderMonkey: Firefox
- JavaScriptCore: Safari

JAVASCRIPT Y NODE.JS

No son lo mismo.

- JavaScript es el lenguaje
- Node.js es un entorno de ejecución para usar JavaScript fuera del navegador

Node.js también trae npm, que permite gestionar paquetes y dependencias.

SCRIPT Y LENGUAJE DE SCRIPTING

Un script es un conjunto de instrucciones que se ejecutan dentro de un entorno.

JavaScript es un lenguaje de scripting de propósito general ajustado a la especificación ECMAScript.

TEMAS TECNICOS VISTOS EN LA PRESENTACION

Además del marco teórico general, el PDF toca:

- variables var, let, const
- tipos de datos primitivos y no primitivos
- objetos
- arrays
- funciones
- funciones flecha
- eventos
- condicionales
- bucles
- DOM
- try...catch

Para teoria, lo importante es saber que son y para que sirven, no necesariamente memorizar sintaxis completa.

4. HERRAMIENTAS Y STACK DE DESARROLLO HIBRIDO

Este bloque viene de herramientas_creacion_proyecto.pdf.

IDEA GENERAL

El desarrollo de aplicaciones moviles hibridas permite construir apps usando tecnologias web como HTML, CSS y JavaScript.

La sesion del material busca:

- instalar herramientas
- crear un proyecto inicial
- entender estructura de archivos y carpetas
- usar consola del navegador
- comprender la importancia de comentar el codigo

VISUAL STUDIO CODE

Visual Studio Code es el editor de texto que se usa para escribir y organizar el codigo.

No es:

- lenguaje de programacion
- framework
- entorno de ejecucion

NODE.JS

Node.js es un entorno de ejecucion que permite ejecutar JavaScript fuera del navegador.

Ademas:

- incluye npm
- permite gestionar dependencias

No es:

- un lenguaje
- un framework visual

ANGULAR

Angular es un framework de desarrollo web basado en TypeScript.

Permite construir aplicaciones dinamicas y estructuradas mediante componentes.

Cada componente puede encapsular:

- estructura (HTML)
- estilo (CSS)
- logica (TypeScript)

Angular se usa para organizar la aplicacion y su logica.

IONIC

Ionic es un framework para desarrollar aplicaciones moviles hibridas.

Su foco principal es:

- interfaz de usuario
- componentes visuales adaptados a mobile

Punto teorico clave:

- Ionic no reemplaza la logica de Angular
- Ionic se ocupa mas de como se ve la app
- Angular se ocupa mas de como funciona

CAPACITOR

Capacitor permite conectar una aplicacion web con funcionalidades nativas del dispositivo.

Ejemplos:

- camara
- GPS
- almacenamiento

Se integra con Angular e Ionic.

COMO SE COMPLEMENTAN

Frase corta muy util:

- Angular organiza la logica
- Ionic aporta la interfaz mobile
- Capacitor conecta con el dispositivo

No compiten entre si. Se complementan.

CLI

El material nombra:

- Angular CLI
- Ionic CLI

Se usan para crear, organizar y ejecutar proyectos.

Comandos mencionados:

- ionic start miProyecto
- cd miProyecto
- ionic serve

ESTRUCTURA DEL PROYECTO

El material remarca que hay que entender la estructura de carpetas y archivos.

Tambien menciona archivos de configuracion como:

- angular.json
- package.json

CONSOLA Y COMENTARIOS

La consola del navegador sirve para:

- ver informacion durante la ejecucion
- depurar
- analizar comportamiento del codigo

Los comentarios sirven para:

- mejorar comprension
- facilitar mantenimiento
- ayudar al trabajo colaborativo

5. HTML

La guía de HTML es práctica, pero deja claros los conceptos que pueden tomarse en teoría.

ESTRUCTURA BASICA

Un documento HTML5 debe incluir:

- <!DOCTYPE html>
- <html>
- <head>
- <body>

ETIQUETAS Y ELEMENTOS VISTOS

- encabezados: <h1> a <h6>
- párrafos: <p>
- salto de línea:

- span
- enlaces: <a>
- imágenes:
- listas: , , <dl>
- tablas: <table>, <tr>, <th>, <td>
- iframe
- formularios
- etiquetas semánticas: <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>
- audio y video

IDEAS TEORICAS IMPORTANTES

- HTML estructura el contenido
- CSS define presentación
- JavaScript agrega comportamiento

FORMULARIOS

El material practica formularios con:

- texto
- contraseña
- número
- radio
- checkbox
- select

También aparece validación HTML5 con atributos como:

- required
- minlength
- maxlength
- placeholder
- min
- max

SEMANTICA

Las etiquetas semanticas mejoran:

- organizacion del documento
- comprension del contenido
- mantenimiento

6. GIT Y GITHUB

La guia es de ejercicios, pero deja muy claro el flujo conceptual que tenes que dominar.

QUE ES GIT

Git es un sistema de control de versiones.

Sirve para:

- registrar cambios
- conservar historial
- trabajar con ramas
- integrar trabajo
- revertir o analizar modificaciones
- colaborar en equipo

QUE ES GITHUB

GitHub es una plataforma remota para alojar repositorios Git y colaborar.

Diferencia corta:

- Git = sistema de versionado
- GitHub = plataforma para compartir y administrar repositorios Git

CONCEPTOS CLAVE

- repositorio: proyecto versionado
- commit: registro puntual de cambios
- historial: secuencia de commits
- branch o rama: linea paralela de trabajo
- merge: integracion de ramas
- conflicto: choque entre cambios incompatibles
- remoto: repositorio alojado fuera del equipo local
- clonar: copiar un repositorio remoto al entorno local

IDEAS FUERTES QUE REMARCAN LOS EJERCICIOS

- los commits deben reflejar el proceso real
- los mensajes de commit deben ser claros y descriptivos
- deben verse las ramas creadas
- debe haber evidencia de merges
- hay que saber resolver conflictos manualmente

FLUJO DE TRABAJO QUE CUBRE LA GUIA

1. Crear carpeta e inicializar repositorio.
2. Crear y modificar archivos.
3. Registrar cambios.
4. Consultar historial.
5. Separar cambios en commits.
6. Crear ramas.
7. Volver a la rama principal.
8. Hacer merge.
9. Crear repo en GitHub.
10. Vincular repo local con remoto.
11. Subir cambios.
12. Clonar repo.
13. Simular trabajo colaborativo.
14. Generar y resolver conflictos.

CONFLICTOS

Un conflicto aparece cuando dos cambios incompatibles afectan la misma parte del proyecto y Git no puede decidir automaticamente cual conservar.

Resolver un conflicto implica:

- revisar las versiones en choque
- decidir que contenido queda
- guardar el archivo corregido
- registrar la resolucion

7. INTEGRACION GENERAL DE TODOS LOS TEMAS

La relacion entre los contenidos se puede estudiar como un flujo unico:

1. Se crea y organiza un proyecto con VS Code, Node.js, Angular, Ionic y Capacitor.
2. La interfaz usa HTML, CSS y JavaScript.
3. La aplicacion consume una API REST.
4. La comunicacion se hace por HTTP.
5. Los datos viajan normalmente en JSON.
6. La autentificacion puede resolverse con JWT.
7. El proyecto se versiona y comparte con Git y GitHub.

Si en el parcial te piden explicar el ecosistema completo, esta cadena te ordena la respuesta.

8. TEMAS CON MAS PROBABILIDAD TEORICA

Si tenes que priorizar:

1. JWT: definicion, estructura, autentificacion, autorizacion, stateless, access token, refresh token, seguridad basica.
2. APIs: API, cliente-servidor, REST, HTTP, JSON, codigos de estado, Postman.
3. JavaScript: que es, origen, ECMAScript, interpretado, tipado dinamico, Node.js.
4. Angular/Ionic/Capacitor: que hace cada uno y como se complementan.
5. Git/GitHub: diferencias, ramas, merge, conflicto, flujo de trabajo.
6. HTML: estructura y etiquetas principales.